作业 2

STM32 与 Cortex-M4 体系结构

彭俊杰　　智装备 241　　2024306100324

1. **说明下列英文缩写的含义：**ARM、AHB、AHB-Lite、APB、PPB、DAP（JTAG、SWD）、MPU、FPU、NVIC、ISR、Bitband、SCS、SysTick、MSP、PC、SP、GPIO、TIM、UART、CANBUS。

表 1　英文缩写与含义

缩写	英文名称	含义
ARM	Advanced RISC Machines	处理器架构及处理器 IP 的名称，采用精简指令集设计。
AHB	Advanced High-performance Bus	AMBA 高性能总线，连接处理器、存储器及高速外设。
AHB-Lite	Advanced High-performance Bus Lite	单主设备 AHB 简化协议；多层矩阵可连接多个主端口。
APB	Advanced Peripheral Bus	面向低带宽外设的简单总线，经桥与 AHB 相连。
PPB	Private Peripheral Bus	内核私有外设总线，访问系统控制、调试等部件。
DAP	Debug Access Port	调试访问端口，由调试端口和访问端口组成。
JTAG	Joint Test Action Group	IEEE 1149.1 边界扫描接口，亦用于处理器调试。
SWD	Serial Wire Debug	串行线调试接口，主要信号为 SWDIO 和 SWCLK。
MPU	Memory Protection Unit	存储器保护单元，按区域设置访问权限和存储属性。
FPU	Floating-Point Unit	浮点运算单元；F407 与 L496 集成单精度 FPU。
NVIC	Nested Vectored Interrupt Controller	嵌套向量中断控制器，管理中断使能、挂起和优先级。
ISR	Interrupt Service Routine	中断服务程序，响应中断时执行的处理函数。
Bitband	Bit-banding	位带机制：别名区的一个 32 位字映射原区域的一位。
SCS	System Control Space	系统控制空间，包含 SCB、NVIC、SysTick 寄存器。
SysTick	System Tick Timer	内核 24 位递减定时器，常用于系统节拍和延时。
MSP	Main Stack Pointer	主栈指针；复位后使用，异常处理模式固定使用。
PC	Program Counter	程序计数器 R15，保存程序执行地址。
SP	Stack Pointer	栈指针 R13；线程模式可选择 MSP 或 PSP。
GPIO	General-Purpose Input/Output	通用输入输出，引脚可配置为输入、输出或复用功能。
TIM	Timer	定时器，支持定时计数、输入捕获、输出比较和 PWM。
UART	Universal Asynchronous Receiver/Transmitter	通用异步收发器；USART 还支持同步通信。
CANBUS	Controller Area Network Bus	控制器局域网总线，采用基于标识符的非破坏性逐位仲裁。

2. 简要画图说明 ARM Cortex-M4 对存储器映射如何划分地址区域的，（自学）标出 GPIO、SysTick、CAN 这些外设寄存器所在的地址在哪个区域？

Cortex-M4 采用 32 位字节地址，地址范围为 `0x00000000`～`0xFFFFFFFF`，共 2^{32} 字节，即 4 GiB。



图 1　Cortex-M4 地址空间分区

表 2　STM32F407 外设地址与所属区域

模块	基址	区域	挂接位置
GPIOA	`0x40020000`	外设区	AHB1
GPIOF	`0x40021400`	外设区	AHB1
CAN1	`0x40006400`	外设区	APB1
CAN2	`0x40006800`	外设区	APB1
SysTick	`0xE000E010`	系统区	PPB 中的 SCS

从总线归属分析：GPIOA 与 GPIOF 挂接在 AHB1 高速外设总线上，CAN 挂接在 APB1 总线上；而 SysTick 属于内核私有外设，直接映射于系统控制空间（SCS，`0xE000E000`～`0xE000EFFF`），经由私有外设总线（PPB，`0xE0000000`～`0xE00FFFFF`）访问，不占用外设总线带宽。SysTick 的 CTRL、LOAD、VAL、CALIB 寄存器地址依次为 `0xE000E010`、`0xE000E014`、`0xE000E018` 与 `0xE000E01C`。此外，Cortex-M4 在 SRAM 区与外设区底部 1 MB 空间分别支持位带别名区映射，便于实现原子位操作。

3. 画简图说明有关中断的概念（罗列关键词），并说明 ARM Cortex-M4 微处理器异常/中断向量表的作用是什么？

Cortex-M4 采用统一的异常管理模型，将内核服务故障与外部中断统筹管理。外设通过 IRQ 线向 NVIC 提交请求，SysTick、SVC、PendSV 则属于内核异常。中断从请求到响应的硬件处理时序见图 2。

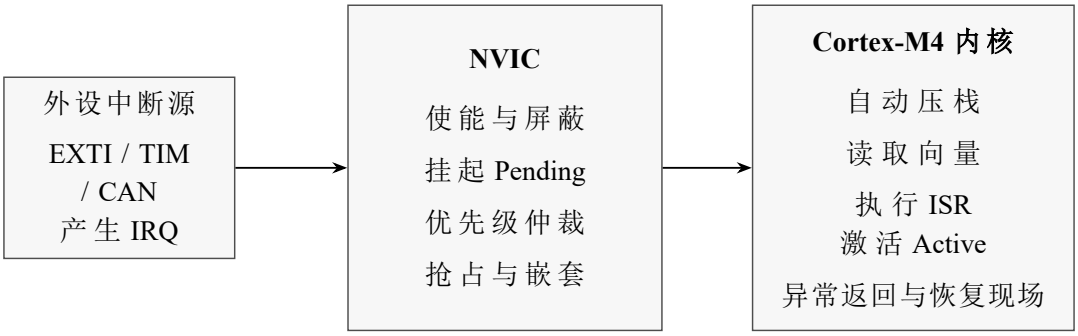


图 2　中断响应过程及关键词

当中断请求满足使能与优先级仲裁条件时，处理器硬件自动在当前堆栈压入基础栈帧（R0～R3、R12、LR、PC 与 xPSR），并由硬件直接从向量表装载 ISR 入口执行。异常返回时，特定的 EXC_RETURN 编码触发硬件自动出栈并恢复现场。编写 ISR 时，须按外设规则显式清除中断挂起标志，防止异常返回后发生错误重入。

异常状态包含未激活、挂起（已记录请求）、激活（正在执行服务）及激活并挂起。可配置优先级的数值越小代表优先级越高；抢占优先级支持高优先级中断打断低优先级执行形成嵌套，子优先级则在抢占优先级相同时决定排队次序。

向量表的作用

向量表将异常编号与处理函数入口对应起来，每项占 4 字节。第 0 项保存初始 MSP，第 1 项保存复位入口；其余表项保存异常入口或保留值。外部中断 IRQn 的入口所在地址为

$$A_{\mathrm{IRQ}} = \mathrm{VTOR} + 4\,(16 + \mathrm{IRQn}).$$

表 3　向量表的主要表项

表项编号	字节偏移	内容
0	`0x00`	初始主栈指针 `__initial_sp`
1	`0x04`	`Reset_Handler`
2 / 3	`0x08` / `0x0C`	`NMI_Handler` / `HardFault_Handler`
4 / 5 / 6	`0x10` / `0x14` / `0x18`	MemManage / BusFault / UsageFault 入口
11 / 14	`0x2C` / `0x38`	`SVC_Handler` / `PendSV_Handler`
15	`0x3C`	`SysTick_Handler`
16 + IRQn	4(16 + IRQn)	外部中断处理函数入口

向量表基址由系统控制块（SCB）中的 VTOR（向量表偏移寄存器）指定，复位时默认为 0 地址。当代码由内部主 Flash 启动时，硬件将 `0x08000000` 映射至 0 地址处；若固件需重定位向量表（如 Bootloader 或 IAP 升级），写入 VTOR 的地址须满足表项对齐要求（通常为 128 字节边界对齐）。

4. 列写 STM32F40x 系列微控制器内部结构中片上外设主从总线有哪些，并说明 GPIO、TIMER2（或简写 TIM2）、USART1 模块分别挂在哪条总线上？

STM32F407 使用 32 位多层 AHB 总线矩阵，连接 8 个主端口和 7 个从端口。片上外设分布在 AHB1、AHB2、AHB3、APB1 和 APB2 上。

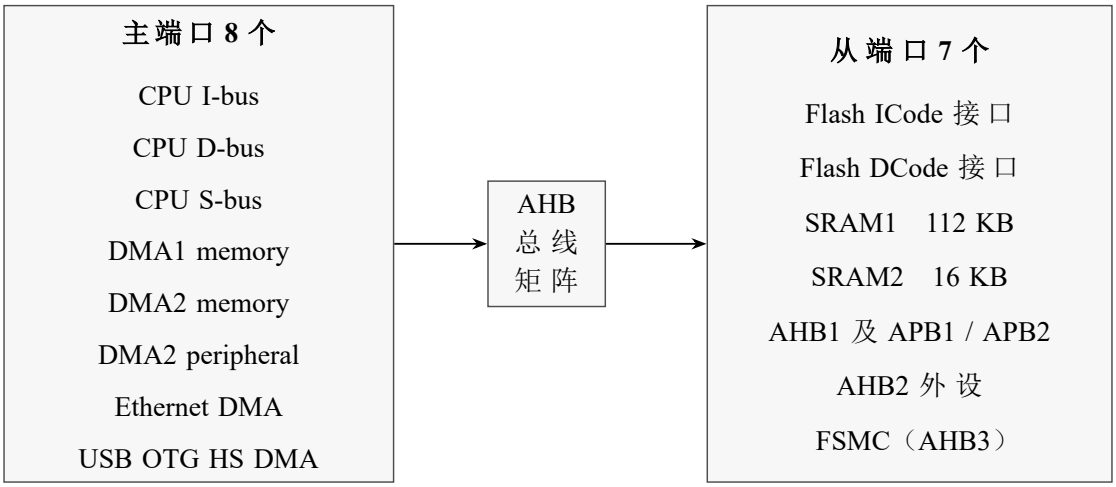


图 3 STM32F407 总线矩阵的主端口与从端口

表 4 GPIO TIM2 USART1 的挂接总线

模块	总线	基址	功能
GPIO	AHB1	GPIOA：0x40020000	通用输入输出
TIM2	APB1	0x40000000	32 位通用定时器
USART1	APB2	0x40011000	同步或异步串行通信

AHB1 经桥连接 APB1 和 APB2；AHB3 连接 FSMC。DMA1 的外设端口直接连接 APB1，存储器端口接入 AHB 总线矩阵。多层 AHB 矩阵采用交叉开关（Crossbar）拓扑，允许多个主设备并发访问不同从设备。特别需要指出的是，64 KB 的 CCM 数据 RAM 独立挂接于内核 D-bus 上，并未接入多层 AHB 矩阵，因此通用 DMA 控制器物理上无法访问 CCM，工程中严禁将外设 DMA 缓冲区分配于 CCM 区域。

5. STM32F40x 系列微控制器的高速和低速时钟源指的是哪些？如何得到系统时钟 SYSCLK？AHB1、APB1、APB2 总线的时钟最高频率各是多少？

表 5　STM32F407 的高速与低速时钟源

类别	名称	频率	用途
高速内部	HSI	16 MHz RC	复位后的 SYSCLK 来源，也可作为 PLL 输入
高速外部	HSE	4～26 MHz 晶体	SYSCLK 或 PLL 输入；本工程晶体为 8 MHz
低速内部	LSI	标称约 32 kHz RC	独立看门狗、RTC
低速外部	LSE	32.768 kHz 晶体	RTC 日历与低速计时

SYSCLK 可选择 HSI、HSE 或主 PLL 的 PLLCLK 输出。主 PLL 由 HSI 或 HSE 驱动，频率关系为

$$f_{\mathrm{VCO}} = \frac{f_{\mathrm{in}}}{M} N, \qquad f_{\mathrm{PLLCLK}} = \frac{f_{\mathrm{VCO}}}{P}, \qquad f_{\mathrm{PLL48CLK}} = \frac{f_{\mathrm{VCO}}}{Q}.$$

取 $f_{\mathrm{in}} = 8\,\mathrm{MHz}$、$M = 8$、$N = 336$、$P = 2$、$Q = 7$，得到 VCO 频率 336 MHz、SYSCLK 168 MHz、PLL48CLK 48 MHz。

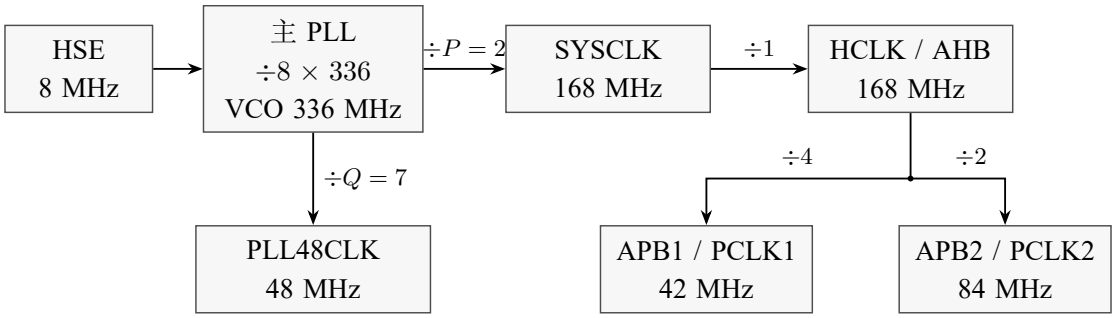


图 4　8 MHz HSE 生成 168 MHz SYSCLK 的配置

表 6　系统与总线时钟的最高频率

时钟	最高频率	本例分频
SYSCLK	168 MHz	主 PLL 输出
AHB1 / HCLK	168 MHz	SYSCLK ÷1
APB1 / PCLK1	42 MHz	HCLK ÷4
APB2 / PCLK2	84 MHz	HCLK ÷2

APB 定时器内部集成了硬件倍频电路：当 APB 预分频系数为 1 时，定时器时钟等于 PCLK；当分频系数大于 1 时，定时器时钟自动配置为 2 × PCLK。本例 APB1 四分频（42 MHz），故 TIM2 等定时器内部时钟为 $2 \times 42 = 84$ MHz，确保了定时器的计数精度。此外，在 168 MHz 满频工作时，需向 FLASH_ACR 寄存器配置 5 个等待周期（`FLASH_LATENCY_5`）并启用指令缓存，以满足供电与读取时序要求。

6. **在 STM32F407ZGT6 芯片引脚上找到复位脚，说明其名称的含义，并画出典型的复位电路。**

STM32F407ZGT6 采用 LQFP144 封装，**第 25 脚为 NRST**。N 表示低电平有效，RST 表示复位。

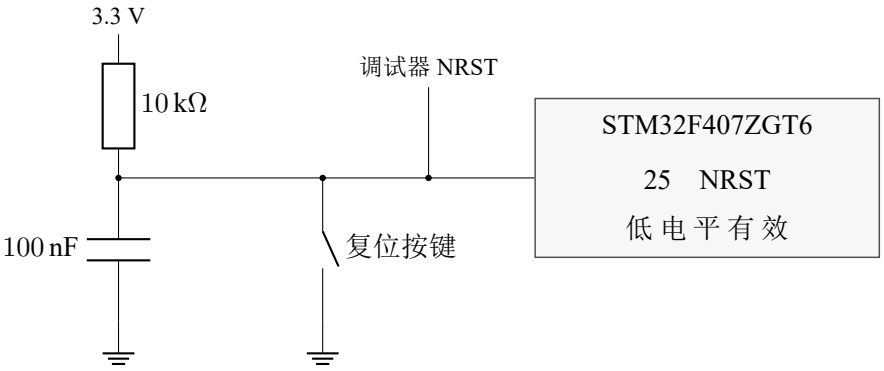


图 5 NRST 复位电路

按键按下时，NRST 接地，芯片复位；松开后，上拉网络使 NRST 恢复高电平。NRST 内部带弱上拉；外部 10 kΩ 电阻加强上拉，100 nF 电容抑制短时干扰。需注意不同封装的引脚差异：查 DS8626 可知，LQFP144 的第 25 脚为 NRST，第 14 脚为 PF4；而 LQFP100 的第 14 脚才是 NRST。此外，LQFP144 的第 143 脚 PDR_ON 在启用内部复位监控时须连接高电平 VDD，不可悬空。

7. **请查找我们实验所用两种开发板（407 通用、潘多拉物联网）说的主芯片 STM32F407ZGT6、STM32L496 的 PDF 资料，简单描述一下其特征。**

STM32F407ZGT6 和 STM32L496VET6 均采用 Cortex-M4 内核，支持 DSP 指令，带单精度 FPU。

项目	STM32F407ZGT6	STM32L496VET6
CPU 最高频率	168 MHz	80 MHz
Flash	1 MB	512 KB
SRAM	192 KB：112 KB SRAM1、16 KB SRAM2、64 KB CCM；另有 4 KB 备份 SRAM	320 KB：256 KB SRAM1、64 KB 带奇偶校验的 SRAM2
封装	LQFP144	LQFP100
工作电源	1.8～3.6 V	1.71～3.6 V
主要接口	Ethernet MAC、USB OTG FS/HS、CAN、USART、SPI、I^2C、FSMC	USB OTG FS、CAN、LPUART、I^2C、SPI、QUADSPI、SDMMC
模拟外设	3 个 12 位 ADC、2 个 DAC 通道	3 个 12 位 ADC、2 个 DAC 通道、运放、比较器
低功耗模式	Sleep、Stop、Standby	Sleep、低功耗运行、低功耗睡眠、Stop 0/1/2、Standby、Shutdown

实验平台辨析：普中 F407-T100 采用 STM32F407ZGT6；正点原子潘多拉开发板实物搭载的是 STM32L475VET6。它与题目所列的同系列低功耗型号 STM32L496VET6 具有相同的 Cortex-M4 内核与封装引脚兼容性，但片上 SRAM 架构存在代差（L475 为 128 KB，L496 扩充为 320 KB）。

8. 扩展练习

① 查阅 STM32F407ZGT6 最小系统电路图

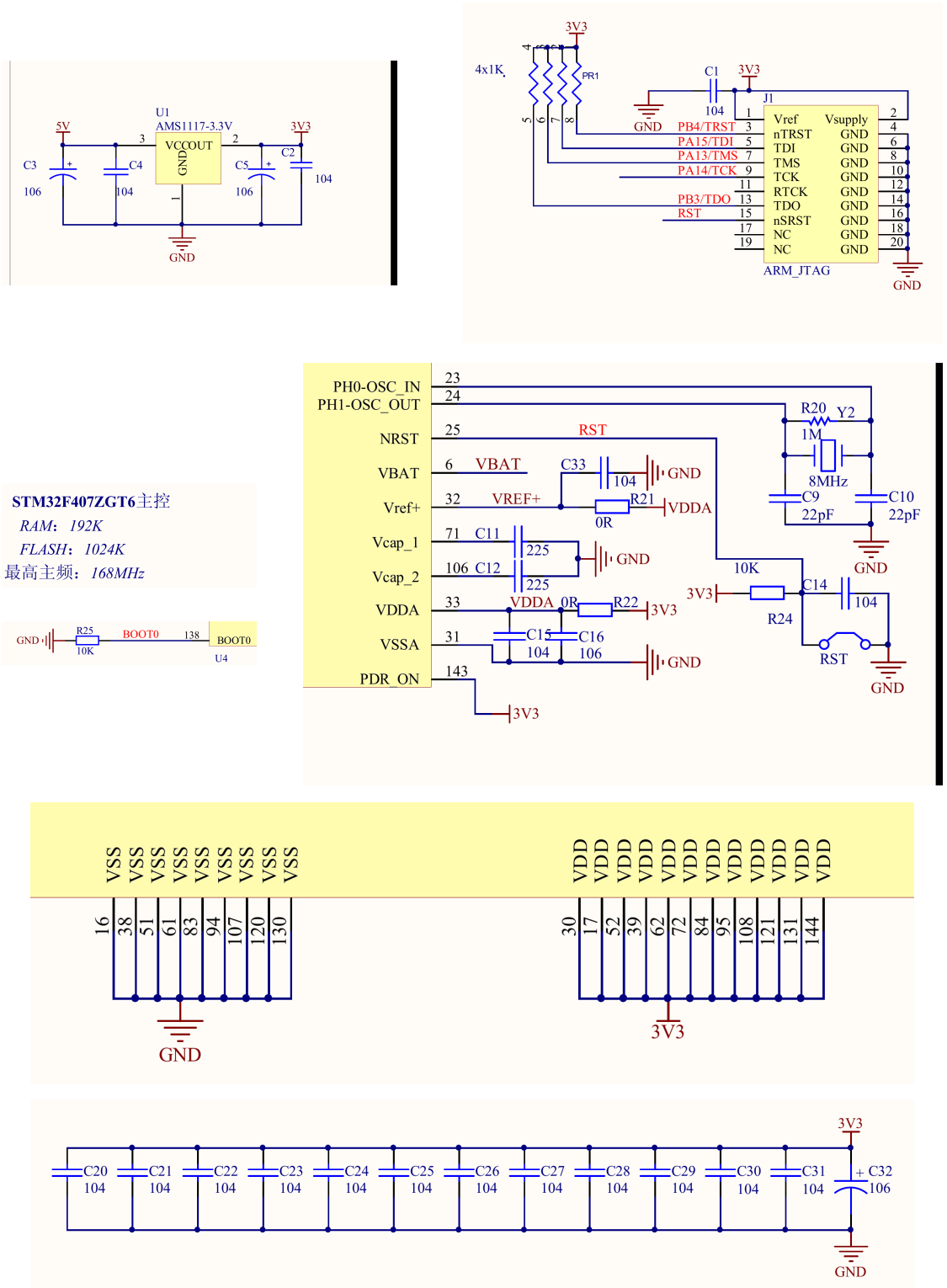


图 6　STM32F407ZGT6 最小系统的供电调试时钟复位与启动电路

STM32F407ZGT6 最小系统涵盖主芯片工作所需的核心支撑电路：包括电源供电滤波、VCAP1/2 稳压滤波端（各外接 2.2 μF 电容）、HSE/LSE 晶振电路、NRST 按键复位、BOOT0 启动电平及 SWD 调试下载接口。同类电源与地引脚须逐一就近接入去耦电容，确保电源完整性。

② 构建标准库与 HAL 库空白工程，理解模板的作用

工程包括 SPL_GPIO 和 HAL_GPIO 两个目录，各有 Blank 空白模板与 Template GPIO 示例。Group 按功能组织源文件；`main.c` 是应用入口；模板统一芯片型号、头文件路径、宏定义、启动文件和链接配置。

表 7　两类工程的组成

组成	标准库 SPL	HAL
用户程序	`main_blank.c` 或 `main.c`，中断处理文件	`main_blank.c` 或 `main.c`，中断处理文件
启动文件	`startup_stm32f40_41xxx.s`	`startup_stm32f407xx.s`
内核与系统	CMSIS、`system_stm32f4xx.c`	CMSIS、`system_stm32f4xx.c`
外设驱动	STM32F4xx 标准外设驱动	STM32F4xx HAL 驱动
空白模板流程	`main` → `while (1)`	`HAL_Init` → `SystemClock_Config` → `while (1)`

③ 分析启动文件中的向量表、堆栈与复位序列

本工程启动文件包含 98 个向量表项：16 个系统表项和 82 个外部中断表项，共 392 字节。`Stack_Size = 0x00000400`，栈为 1 KB；`Heap_Size = 0x00000200`，堆为 512 B。DCD 按 32 位字放置表项。

```
__Vectors       DCD     __initial_sp
                DCD     Reset_Handler
                DCD     NMI_Handler
                DCD     HardFault_Handler

Reset_Handler   PROC
                EXPORT  Reset_Handler [WEAK]
                IMPORT  SystemInit
                IMPORT  __main
                LDR     R0, =SystemInit
                BLX     R0
                LDR     R0, =__main
                BX      R0
                ENDP
```

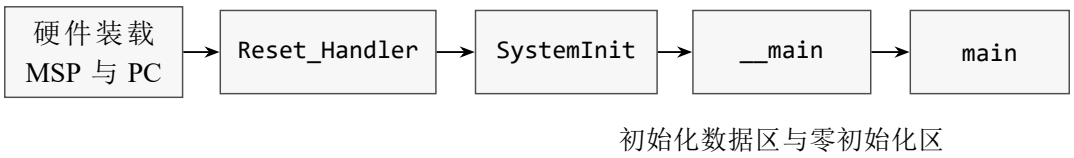


图 7　从复位到 C 程序入口的执行顺序

从主 Flash 启动时，芯片将 Flash 映射到地址 0。处理器从 `0x00000000` 读取初始 MSP，从 `0x00000004` 读取复位入口。`SystemInit` 完成内核与系统初始化；C 运行库 `__main` 初始化数据区后调用 `main`。

④ 新建 GPIO 工程，体会标准库与 HAL 库的使用

PF9 接低电平点亮的 LED，PE4 接按下接地的按键。PF9 配置为推挽输出，PE4 配置为上拉输入；按键按下时输出低电平，LED 点亮。

标准库程序

```
#include "stm32f4xx.h"

static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio;
    RCC_AHB1PeriphClockCmd(
        RCC_AHB1Periph_GPIOF | RCC_AHB1Periph_GPIOE, ENABLE);

    GPIO_SetBits(GPIOF, GPIO_Pin_9);
    GPIO_StructInit(&gpio);
    gpio.GPIO_Pin = GPIO_Pin_9;
    gpio.GPIO_Mode = GPIO_Mode_OUT;
    gpio.GPIO_OType = GPIO_OType_PP;
    gpio.GPIO_Speed = GPIO_Speed_2MHz;
    gpio.GPIO_PuPd = GPIO_PuPd_NOPULL;
    GPIO_Init(GPIOF, &gpio);

    gpio.GPIO_Pin = GPIO_Pin_4;
    gpio.GPIO_Mode = GPIO_Mode_IN;
    gpio.GPIO_PuPd = GPIO_PuPd_UP;
    GPIO_Init(GPIOE, &gpio);
}

int main(void)
{
    GPIO_Setup();
    while (1)
    {
        if (GPIO_ReadInputDataBit(GPIOE, GPIO_Pin_4) == Bit_RESET)
            GPIO_ResetBits(GPIOF, GPIO_Pin_9);
        else
            GPIO_SetBits(GPIOF, GPIO_Pin_9);
    }
}
```

表 8　按键输入与 LED 输出关系

按键状态	PE4 输入	PF9 输出	LED 状态
松开	高电平	高电平	熄灭
按下	低电平	低电平	点亮

HAL 初始化与主循环

```c
#include "main.h"

static void SystemClock_Config(void);
static void GPIO_Setup(void);

int main(void)
{
    HAL_Init();
    SystemClock_Config();
    GPIO_Setup();
    while (1)
    {
        GPIO_PinState key = HAL_GPIO_ReadPin(GPIOE, GPIO_PIN_4);
        HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, key);
    }
}

static void GPIO_Setup(void)
{
    GPIO_InitTypeDef gpio = {0};
    __HAL_RCC_GPIOF_CLK_ENABLE();
    __HAL_RCC_GPIOE_CLK_ENABLE();

    HAL_GPIO_WritePin(GPIOF, GPIO_PIN_9, GPIO_PIN_SET);
    gpio.Pin = GPIO_PIN_9;
    gpio.Mode = GPIO_MODE_OUTPUT_PP;
    gpio.Pull = GPIO_NOPULL;
    gpio.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOF, &gpio);

    gpio.Pin = GPIO_PIN_4;
    gpio.Mode = GPIO_MODE_INPUT;
    gpio.Pull = GPIO_PULLUP;
    HAL_GPIO_Init(GPIOE, &gpio);
}
```

表 9　标准库与 HAL 的 GPIO 函数

操作	标准库 SPL	HAL
使能时钟	RCC_AHB1PeriphClockCmd	__HAL_RCC_GPIOx_CLK_ENABLE
初始化	GPIO_Init	HAL_GPIO_Init
读取引脚	GPIO_ReadInputDataBit	HAL_GPIO_ReadPin
写入引脚	GPIO_SetBits / GPIO_ResetBits	HAL_GPIO_WritePin

在底层硬件机制上：读引脚函数通过输入数据寄存器（IDR）获取引脚物理电平；写引脚函数则借助端口置位/复位寄存器（BSRR）实现原子操作。BSRR 属于只写命令寄存器（读操作恒返回 0），低 16 位写 1 置高，高 16 位写 1 置低，写 0 保持不变，单次 32 位总线写操作即可完成电平翻转，规避了对输出数据寄存器 ODR 执行读-改-写（|= 或 &=）可能引发的中断竞态风险。此外，配置结构体中的输出速度控制的是推挽 MOS 管的压摆率（Slew Rate），用于抑制高频辐射干扰，其物理特性独立于软件逻辑翻转频率。